

UD 1. INTRODUCCIÓN A LA PROGRAMACIÓN

**Programación – 1º DAM
Curso 2025/2026**

Índice

1.Introducción.

2.Resolución de problemas.

2.1.Algoritmos y programas.

3. Paradigmas de la programación.

4. Fases de la programación.

4.1.Fase de resolución del problema.

4.2.Fase de implementación.

4.3.Fase de explotación y mantenimiento.

5. Ciclo de vida del software.

1.Introducción

- Si nos detenemos a pensar un instante nos daremos cuenta de que casi todo lo que nos rodea, en alguna medida, está relacionado con la programación, los programas y el tratamiento de algún tipo de información.
- Vamos a aprender los conceptos fundamentales de la programación de aplicaciones. Veremos con qué vamos a trabajar, qué técnicas podemos emplear y cuál es el objetivo que queremos conseguir.
- Veremos los tipos de programación más relevantes, identificaremos las fases de desarrollo de un programa.
- También conoceremos las características relevantes de los distintos lenguajes de programación.
- Por último, veremos qué herramientas necesitamos para desarrollar nuestros programas.

2.Resolución de problemas

- Normalmente, la razón la primera razón que mueve a una persona a hacer un programa es resolver problemas concretos de forma rápida y eficiente. Otro motivo puede ser la automatización de tareas.
- Cuando se trata de resolver problemas en la vida real, se pasa por diferentes fases. Estas fases se corresponden a su vez con las diferentes etapas del desarrollo de un programa

Vida Real	Programación
Observación de la situación o problema.	Análisis del problema: requiere que el problema sea definido y comprendido claramente para que pueda ser analizado con todo detalle.
Pensamos en una o varias posibles soluciones.	Diseño o desarrollo de algoritmos: procedimiento paso a paso para solucionar el problema dado. Aplicamos la solución que estimamos más adecuada.
Aplicamos la solución que estimamos más adecuada.	Implementación del programa: consiste en convertir el algoritmo en programa, ejecutarlo y comprobar que soluciona verdaderamente el problema.

2.Resolución de problemas

- La solución ha de ser:
 - Correcta: que no tiene fallos.
 - Eficaz: si soluciona el problema planteado de forma adecuada.
 - Eficiente: si lo hace en poco tiempo y usa de forma óptima los recursos del sistema.
- Para ello, nuestra solución se habrá diseñado teniendo en cuenta los siguientes conceptos:

Abstracción	se descompone el problema principal en problemas más pequeños y sencillos. Divide y vencerás , es una técnica de resolución de problemas que se usa para abstraer.
Encapsulación	consiste en ocultar cómo se ha resuelto el problema permitiendo que se pueda usar la solución sin conocer el cómo se ha resuelto.
Modularidad	Se debe estructurar cada subproblema en módulos independientes, cada uno con su función específica, de forma que unos puedan usar otros y así componer la solución final.

2.1. Algoritmos y programas

- Un **algoritmo** es una secuencia ordenada de pasos, descrita sin ambigüedades, que conducen a la solución de un problema dado. Es como una receta concreta y precisa para solucionar el problema.
- Los algoritmos son independientes de los lenguajes de programación y de los equipos donde se ejecutan. Un mismo algoritmo puede ser expresado en diferentes lenguajes de programación y podría ser ejecutado en diferentes dispositivos.
- La diferencia fundamental entre algoritmo y programa es que, en un **programa**, los pasos que permiten resolver el problema deben escribirse en un determinado lenguaje de programación para que puedan ser ejecutados en el ordenador.
- Los **lenguajes de programación** son un medio para describir el algoritmo y el ordenador un procesador para ejecutarlo. El diseño de los algoritmos es una tarea que necesitará de creatividad y conocimientos de técnicas de programación. Cada programador tiene su estilo de forma que varios profesionales darán lugar a algoritmos diferentes, y todos serán igualmente válidos.

2.1. Algoritmos y programas

Características de los algoritmos

Como ya hemos visto, un algoritmo es una secuencia de pasos finitos bien definidos que permiten resolver un problema

Para que esto sea así, los algoritmos deben ser:

- Precisos: no deben presentar ambigüedades.
- ordenados: deben indicar la secuencia de realización de las tareas paso a paso.
- definidos: si se ejecuta dos o más veces, debe obtener el mismo resultado cada vez y no debe ejecutar operaciones que no estén reflejadas en él.
- finitos: debe tener un número finito de pasos.

2.1. Algoritmos y programas

Representación de algoritmos

Para representar gráficamente los algoritmos que vamos a diseñar, existen diferentes herramientas que ayudarán a describir su comportamiento de una forma precisa y genérica, para luego poder codificarlos con el lenguaje que nos interese. Algunos ejemplos de estas herramientas son:

- **Diagramas de flujo:** esta técnica utiliza símbolos gráficos para la representación del algoritmo. Suele utilizarse en las fases de análisis.
- **Pseudocódigo:** esta técnica se basa en el uso de palabras clave en lenguaje natural, constantes, variables, otros objetos, instrucciones y estructuras de programación que expresan de forma escrita la solución del problema. Es la técnica más utilizada actualmente.
- **Tablas de decisión:** es una tabla en la que aparecen representadas las posibles condiciones y decisiones del problema con sus las acciones asociadas a cada una de ellas. Se suele usar como apoyo al pseudocódigo, sobre todo cuando existen situaciones condicionales complejas.

2.1. Algoritmos y programas

Ejemplo de algoritmo

Pongamos como ejemplo la realización de el algoritmo para freír un huevo.

- Datos de entrada: Huevo, aceite, sartén, fuego.
- Datos de salida: huevo frito.
- Procedimiento:
 1. Poner el aceite en la sartén.
 2. Poner la sartén al fuego.
 3. Cuando el aceite esté caliente, cascar el huevo e introducirlo.
 4. Cubrir el huevo de aceite.
 5. Cuando el huevo esté hecho, retirarlo.

3. Paradigmas de la programación

- Cuando nos enfrentamos a un problema, se no pueden ocurrir varias formas de resolverlo.
- Un **paradigma** de programación es un modelo básico para el diseño y la implementación de programas. Determina fundamentalmente cómo será el proceso de diseño y la estructura final del programa.
- Cada paradigma tiene sus ventajas e inconvenientes, es más o menos apropiado para solucionar determinado tipo de problemas, pero esto no quiere decir que exista uno mejor que los demás. Es una decisión de diseño la que nos hace usar un paradigma u otro. El hecho de elegir correctamente el paradigma nos permite:
 - facilitar el mantenimiento de las aplicaciones.
 - mejorar el rendimiento del programador.
 - mejorar la eficiencia y calidad de los programas.

En este enlace encontrarás información sobre los distintos tipos de paradigmas:

<https://javierleal.wordpress.com/2009/08/27/paradigmas-de-programacion/>

4. Fases de la programación

El proceso de creación de software puede dividirse en diferentes fases:

1. Fase de resolución del problema.
2. Fase de implementación.
3. Fase de explotación y mantenimiento.

4.1. Fase de resolución del problema.

Para poder solucionar el problema, debe estar perfectamente definido para poder entenderlo y analizarlo.

Esta fase consta, a su vez, de dos fases:

- Fase de **análisis**: en esta fase se averiguan todas las necesidades que debe cubrir el programa. Se realizan entrevistas con los clientes y a veces se desarrollan prototipos para una mejor comprensión del problema. El análisis inicial ofrecerá una idea general de lo que se solicita, realizando posteriormente sucesivas mejoras hasta llegar a la solución.

El resultado de este proceso es la definición de los procesos y estructuras de datos que se van a emplear, los datos que se proporcionan como inicio al programa y los resultados que debe proporcionar tras su ejecución.

En esta fase se debe analizar la documentación proporcionada por el cliente, investigar, observar todo lo que rodea al problema y recopilar cualquier información relacionada con él.

4.1. Fase de resolución del problema

- Fase de **diseño**: En esta etapa se transforma la especificación obtenida en la fase de análisis en un diseño más detallado, indicando la secuencia lógica de instrucciones capaz de resolver el problema planteado. Esto es, como ya hemos visto, el algoritmo.

Una vez se tiene la secuencia, se trata de ver si se puede descomponer la solución global en operaciones más sencillas, detalladas y específicas. Cada una de estas operaciones irán incluidas en módulos separados.

Antes de implementar un algoritmo, hay que asegurarse de que la solución es adecuada. Para ello, se realizarán pruebas. Normalmente se realiza un seguimiento paso a paso de las instrucciones del algoritmo utilizando datos concretos. Si la solución aportada tiene errores, tendremos que volver a la fase de análisis para realizar las modificaciones necesarias o tomar un nuevo camino para la solución. Sólo cuando el algoritmo cumpla los requisitos y objetivos especificados en la fase de análisis se pasará a la fase de implementación.

4.2. Fase de implementación.

En esta fase se le da vida al diseño que hemos realizado en la fase previa. Esta fase se suele realizar en dos etapas:

- **Codificación o construcción.**

En esta etapa se ‘traducen’ los resultados obtenidos a un determinado lenguaje de programación. Además, para comprobar la calidad del programa se han de realizar una serie de pruebas que comprueben las funciones de cada módulo (pruebas unitarias), que los módulos funcionan bien entre ellos (pruebas de interconexión) y que todos funcionan en conjunto correctamente (pruebas de integración).

Cuando realizamos esta traducción del algoritmo al lenguaje de alto nivel, teniendo en cuenta las reglas gramaticales y la sintaxis del mismo, obtendremos lo que se denomina **código fuente**. Esto es lo que normalmente conocemos por programa.

este lenguaje de programación es un lenguaje de alto nivel, y el procesador del ordenador no lo ‘entiende’. Para poder ejecutar el programa, se necesita de una segunda traducción, que puede hacerse de dos formas, compilando o interpretando el código fuente del programa.

4.2. Fase de implementación.

- **Prueba de ejecución y validación..**

Para esta etapa es necesario implantar la aplicación en el sistema donde va a funcionar, debe ponerse en marcha y comprobar si su funcionamiento es correcto. Utilizando diferentes datos de prueba se verá si el programa responde a los requerimientos especificados, si se detectan nuevos errores, si éstos son bien gestionados y si la interfaz es 'user-friendly'. Se trata de poner a prueba nuestro programa para ver su respuesta en cualquier situación que se nos pueda ocurrir.

Mientras se detecten errores y éstos no se arreglen, no podremos avanzar a la siguiente fase. Una vez corregido el programa y testeado se documentará.

4.2. Fase de explotación y mantenimiento.

Cuando el programa ya está instalado en el sistema y está siendo manejado por los usuarios, decimos que se encuentra en fase de **explotación**.

Periódicamente será necesario realizar evaluaciones y, si es necesario, llevar a cabo modificaciones para que el programa se adapte o actualice a nuevas necesidades, pudiendo también corregirse errores no detectados anteriormente. Este proceso recibe el nombre de **mantenimiento** del software.

El mantenimiento del software se define como el proceso de mejora y optimización del software después de su entrega al usuario final. Involucra cambios al software para corregir defectos y dependencias encontradas durante su uso, así como la adición de nuevas funcionalidades para mejorar su usabilidad y su aplicabilidad.

5. Ciclo de vida del software

Siempre que se va a realizar la creación de un programa, sean cuales sean las fases en las que vayamos a realizar el proceso de desarrollo de software, se debe aplicar un modelo de ciclo de vida.

Un ciclo de vida del software es una sucesión de estados o fases por las cuales pasa un software a lo largo de su "vida".

Existen diferentes modelos de ciclo de vida, pero todos ellos han de pasar por estas etapas:

1. Especificación y Análisis de requisitos.
2. Diseño.
3. Codificación.
4. Pruebas.
5. Instalación y paso a Producción.
6. Mantenimiento.